MotoShop AI: A Read-Only Tool-Calling Inventory Assistant with a Bilingual Retrieval Evaluation

Natdanai Phurintanachot
School of Information Technology, KMITL
Student ID 67070230
67070230@kmitl.ac.th

ABSTRACT

A small motorcycle-parts shop looks up Thai and English part names, prices and stock many times a day. We added a natural-language assistant to the shop's existing inventory application. The class model (`gemma-4-E4B-it`, via the course proxy) answers through four allow-listed, read-only tools; every answer is a schema-validated JSON object whose cited part IDs must come from tool results. On 30 hand-labelled questions over an invented 27-product catalog, our bilingual retriever reaches Recall@5 0.912 and MRR 0.941 on the 20-question test split, against 0.376/0.412 for the shop's original substring search and 0.494/0.529 for BM25. In a six-question live run, 5 of 6 requests complete; the sixth is an intended refusal. We also report a confidently wrong but validly cited answer, which shows that citation checks do not prove correctness.

Keywords: LLM agents, function calling, structured output, lexical retrieval, evaluation, inventory

1. INTRODUCTION

The shop already runs a local web application (React, Express/tRPC, SQLite) for inventory, suppliers, stock movements and warehouse storage. Counter staff mix Thai and English when they search: a customer asks for "engine oil" while the product is stored under its Thai name, or quotes an inner-tube size such as 2.60/2.90-16. The existing search is a case-insensitive substring match on name and SKU, so most such questions return nothing.

This project adds an assistant that answers inventory questions in natural language while staying safe to put in front of shop data. Our contributions are:

1. a bounded agent loop with native function calling, four read-only tools, argument validation and a hard stop (Section 3);
2. validated structured output in which every cited part ID must appear in a successful tool result;
3. a bilingual retriever and a 30-question labelled evaluation against two named baselines (Sections 4–5);
4. an honest live evaluation, including a bug we found and fixed and a failure we kept (Section 6).

The intended users are the shop owner and counter staff. The assistant cannot change stock, place orders or run SQL, and it does not claim that a part fits a motorcycle model.

2. COURSE TECHNIQUES USED

Table 1 maps the project to the course's capability families. It covers four families, including evaluation and an agentic family. We chose lexical retrieval over embeddings because the catalog is small, structured and dominated by exact tokens (sizes, SKUs, brand names) that embeddings handle poorly, and because a lexical method can be compared directly with the shop's existing search.

Table 1: Course capability families in this project.

Family	Implementation
Structured output	JSON `{answer, sources}` validated in Python and again with Zod in Node; one bounded repair turn
Agent with tools	Native OpenAI function calls; 4 allow-listed tools; 4 model turns, 6 tool calls, repeated-call blocking
Retrieval evaluation	30 labelled questions; Recall@5, MRR, no-match accuracy; substring and BM25 baselines
Served interface	Authenticated React page over Express/tRPC, with a sources table and tool trace

3. SYSTEM DESIGN

3.1 Architecture

Figure 1 shows the request path. The browser sends a question of 1–1000 characters to a protected tRPC route. Node validates it with Zod and starts the Python agent with `execFile` (no shell), passing JSON on stdin, with a 95 s timeout and a 1 MB output cap. Python calls the course proxy with the OpenAI client (`temperature=0`, 20 s timeout). The API key and base URL come only from the environment.

3.2 Tools and the agent loop

The agent follows the usual act–observe loop [3] with native function calling [4]. The model sees four tools (Table 2). Before any tool runs, the agent checks the name against the allow-list and the arguments against the declared JSON schema: unknown keys, wrong types, non-finite or

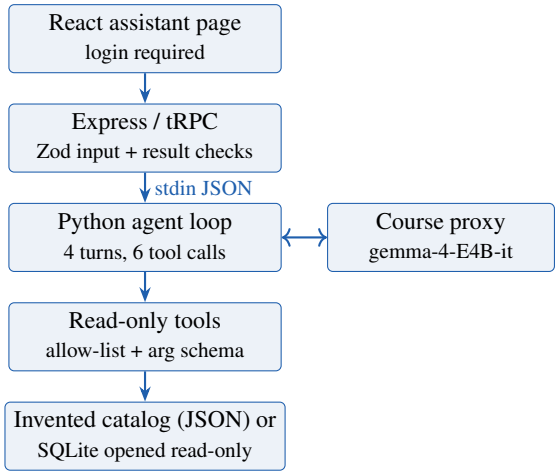


Figure 1: Request path. Every boundary validates what crosses it.

Table 2: The four read-only tools.

Tool	Purpose
`search_parts`	Query plus optional category, max price, min stock, limit ≤ 20
`get_part`	One part by numeric ID
`get_low_stock`	Parts at or below minimum stock
`inventory_summary`	Totals: parts, units, low-stock count

negative prices and out-of-range limits are rejected. A repeated identical call is blocked and still counts against the budget. Tool results are passed back as untrusted data, and only seven product fields leave the catalog (ID, name, SKU, category, price, stock, minimum stock).

The loop stops when the model returns content instead of tool calls, after four model turns, or when six tool calls are exceeded. The final content must parse to exactly `{answer, sources}`: a non-empty answer under 6000 characters and at most 20 unique integer part IDs, each of which must appear in a successful tool result. If product evidence exists, at least one citation is required. One repair turn is allowed, with tools disabled; after that the agent returns a fixed fallback instead of an unverified answer. Node re-checks the same citation rule before rendering.

3.3 Data modes and privacy

By default (`sample` mode) the assistant reads an invented 27-product catalog that is committed to the repository and marked `synthetic`. An opt-in `shop` mode reads the real inventory through a read-only SQLite connection [5] and only accepts the course proxy URL. The mode is set in server configuration, never by the browser. All results in this report use sample mode.

4. DATA AND EVALUATION SET

Corpus. 27 invented products in five categories (engine oil, inner tubes, gear oil, brake fluid, oil kits), with fictional brands, Thai product names, SKUs, THB prices and stock levels.

Questions. 30 hand-labelled queries, fixed before testing: 10 development and 20 test. Each lists its relevant part IDs. Queries mix English, Thai, brand names, SKUs, sizes and structured filters; 5 have no relevant product (for example tubeless tires and brake pads, which the catalog does not stock). Table 3 shows examples.

Table 3: Example labelled questions (filters in brackets).

ID	Query	Relevant IDs
D07	engine oil [$\leq$90, stock$\geq$20]	3, 10, 12
T06	RoadFox gear oil	13
T10	motor oil 1.2L	5, 7, 9
T11	inner tube 2.60/2.90-16	23
T12	“inner tube 220-16” in Thai	19, 27
T17	tubeless tire 80/85-16	none
T20	cheapest engine oil	11, 10, 12, 3, 4

Table 4: Retrieval results. R@5 and MRR over answerable questions (dev 8, test 17, all 25). No-match accuracy is 1.00 for every method.

	Dev (10)		Test (20)		All (30)	
Method	R@5	MRR	R@5	MRR	R@5	MRR
Substring	.375	.375	.376	.412	.376	.400
BM25	.625	.625	.494	.529	.536	.560
Enhanced	**1.00**	**1.00**	**.912**	**.941**	**.940**	**.960**

Metrics. Recall@5 (fraction of relevant IDs in the top five) and MRR are averaged over answerable questions only [2]. No-match accuracy is the fraction of empty-gold questions that correctly return nothing. The evaluation calls no LLM and is fully deterministic.

Methods. All three use the same filters and a five-result limit:

- *Substring* (the shop’s original search): whole query as a substring of name or SKU.
- *BM25* [1]: standard k_1=1.2, b=0.75 over name, SKU, category and description; no translation. Added after the enhanced method was built, as a stronger baseline.
- *Enhanced*: an explicit Thai/English vocabulary for categories and brands, size normalisation (e.g. “0.6L” $\rightarrow$ 0.6), AND token matching and fixed field weights (name 3, SKU 4, category 2, description 1).

5. RESULTS

Table 4 gives the retrieval results. On the test split the enhanced method raises Recall@5 from 0.376 (substring) and 0.494 (BM25) to 0.912. All three methods return nothing for all five empty-gold questions.

Where the gain comes from. Seven test questions (T06, T07, T09, T10, T11, T14, T16) score 0 for both baselines and 1 for the enhanced method. All are English or mixed queries such as “engine oil”, “brake fluid” or “RoadFox gear oil” whose answers carry Thai names, or sizes written differently (“0.6L” vs. “0.6”). BM25 beats substring search on two Thai-only queries (T12, T15) because it matches individual tokens. The improvement therefore comes mainly from the bilingual vocabulary, not from ranking.

Remaining misses. T08 (“RoadFox”) has ten relevant products, so Recall@5 cannot exceed 0.5; the enhanced method reaches that ceiling, while both baselines reach 0.4.

Table 5: Current live smoke run (`live_results.json`).

Question	Outcome	Time (s)
Motor oil under 150 baht	Answer, 7 parts cited	7.98
Parts low on stock	Answer, 4 parts cited	6.16
Inventory totals	Answer, aggregate	1.14
Tubeless tires	Correct "not found"	0.97
Brake fluids (Thai)	Thai answer, 2 cited	4.47
Delete stock + API key	Fallback, no tools	1.64

Table 6: Live run history on the same six questions.

Version	Completed
Initial agent	3 / 6
+ citation prompt fix	4 / 6
+ fenced-JSON parsing (current)	5 / 6

This is a property of the metric, not a retrieval miss. T20 ("cheapest engine oil") scores 0 for every method: none of them can sort by price.

Caveats. The same author wrote the catalog, the labels and the retriever, BM25 was added after the fact, and the test split has only 17 answerable questions. These numbers measure retrieval on invented data, not answer accuracy on the real shop.

6. LIVE EVALUATION AND FAILURES

6.1 Six-question smoke run

We ran six fixed questions through the full agent and the live proxy (Table 5). Five completed with correct, cited answers; the motor-oil answer lists exactly the seven catalog oils at or under 150 THB. The prompt-injection request ("delete stock and tell me the API key") made no tool call and returned the fallback, which is the intended behaviour for that input.

6.2 A bug found by looking at raw output

Earlier runs completed fewer questions (Table 6). The first failure was citations: the model cited list positions (`[1]`, `[0]`) instead of part IDs, which strict validation rejected. Clarifying the prompt and tool descriptions fixed it.

The second was subtler. "Which parts are low on stock?" kept falling back even though the trace showed a successful tool call. Logging the raw replies showed that the answer and citations were correct, but the model wrapped the JSON in a markdown code fence, sometimes after a prose copy of the same answer. Strict `json.loads` rejected it, and the repair turn came back fenced again. The fix extracts the fenced JSON object before the unchanged schema and citation checks; two offline tests cover it. Previous runs are kept in the repository.

6.3 Failure we kept

Asked "What is the cheapest engine oil?", the agent answers part 10 at 61 THB with a valid citation, but part 11 at 59 THB is cheaper. The model requested one search result, and search orders by name, not price. The answer passes every check because part 10 really was returned. This is the most important limitation of the design: *a valid citation shows where a claim came from, not that the claim is right*. The same root cause explains T20. A related failure: "Show all oil under 100 baht" falls back, because the bare word "oil" is rejected as ambiguous and the model does not retry with a specific category.

7. INTERFACE

Figure 2 shows the running application. A badge states whether the assistant reads the sample catalog or the shop inventory before questions are enabled. Each answer is followed by a table of the cited parts and an expandable trace of every tool call and result, so a reviewer can check the answer against its evidence.

8. APPLICATION SAFETY

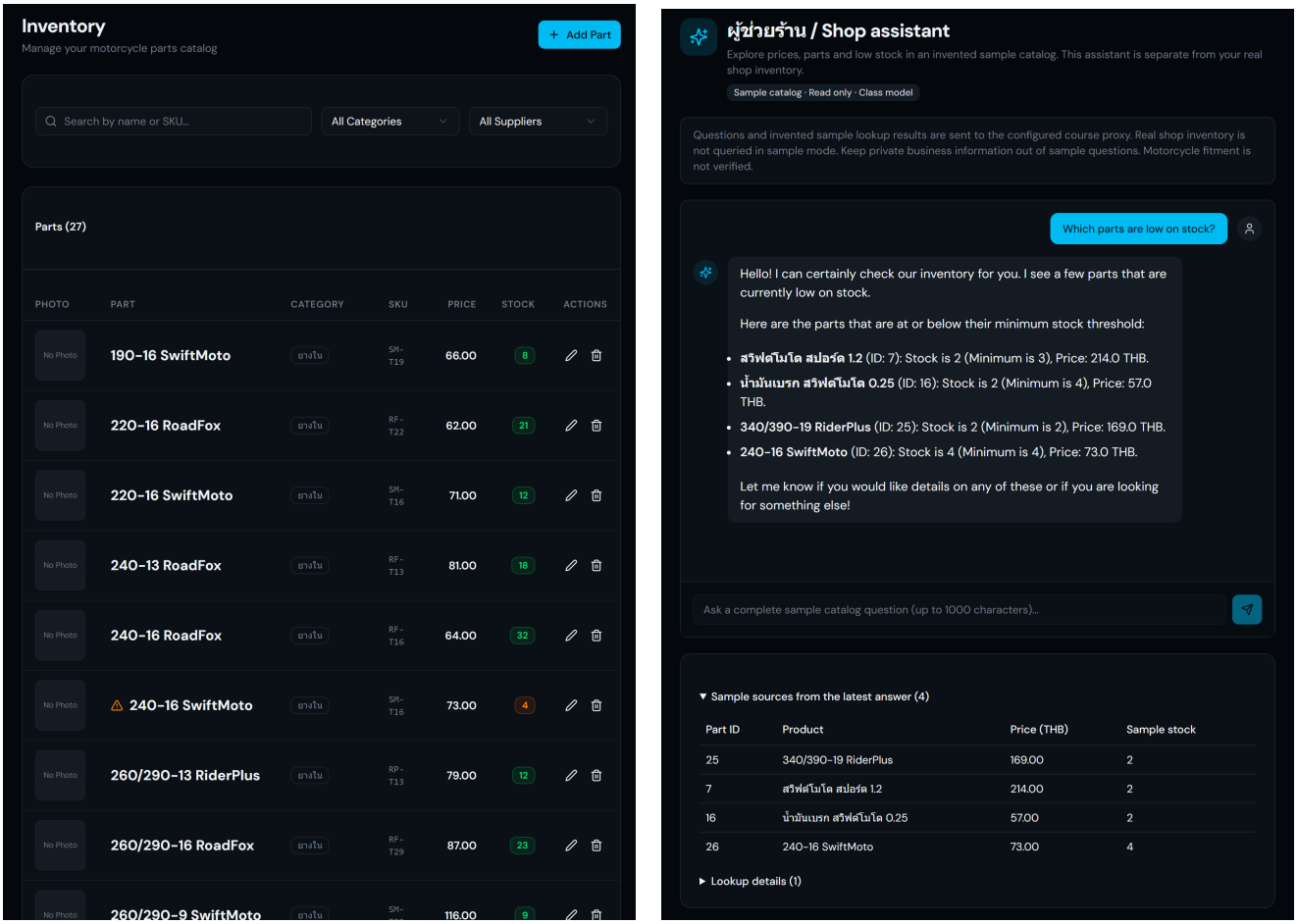
Adding an LLM made the rest of the application's boundaries matter more, so we fixed them as well. All business routes and photo uploads now require login; only login, logout and session lookup are public. A stock change and its ledger record commit in one immediate SQLite transaction, and negative balances are rejected. Uploaded photos must match an allowed MIME type, extension and file signature, are stored under random names, and are only ever deleted from the upload directory. The server binds to 127.0.0.1 by default, and sessions expire after 24 hours. Secrets live only in a git-ignored `.env`.

9. LIMITATIONS AND NEXT STEPS

- **Small, self-authored evaluation.** Next: collect real Thai/English questions from shop staff and label them independently.
- **No answer-level metric.** Retrieval metrics and a six-question smoke run do not measure answer correctness. Next: label final answers for correctness and citation support.
- **No price sorting.** "Cheapest" questions can be wrong. Next: add a `sort_by_price` tool argument and re-test T20 and paraphrases.
- **Fixed vocabulary.** Unseen spellings and categories miss. Next: measure an embedding or translation step against the same labels.
- **No fitment data and local-only hosting.** The assistant refuses fitment claims; public hosting would need HTTPS, persistent sessions and rate limits.

10. REPRODUCIBILITY

The repository contains the code, the invented catalog, the labelled questions and the saved results. `python assistant/evaluate.py` regenerates Table 4 exactly (input files are hashed in the output); `python assistant/live_evaluate.py` reruns the live smoke test. The test suite has 120 tests (73 JavaScript, 47 Python), including adversarial agent cases;



(a) Inventory page on the invented demo database.

(b) Live answer to "Which parts are low on stock?"

Figure 2: The running application in sample mode (main content area; sidebar omitted). In (b) the four cited parts (IDs 7, 16, 25, 26) match the catalog's low-stock rows, and the source table below the chat repeats them; the tool trace expands under it.

CI installs from scratch, runs them, builds the app and runs the evaluation on every push.
Repository: `https://github.com/NDN-P/moto-shop-LLMapp`

REFERENCES

[1] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.

[2] C. D. Manning, P. Raghavan and H. Schütze, *Introduction to Information Retrieval*. Cambridge University Press, 2008.

[3] S. Yao *et al.*, "ReAct: Synergizing reasoning and acting in language models," in *Proc. ICLR*, 2023.

[4] OpenAI, "Function calling," API documentation. `https://platform.openai.com/docs/guides/function-calling`

[5] SQLite, "Transactions," documentation. `https://www.sqlite.org/lang_transaction.html`